

MANUAL TÉCNICO

GREEN BIT RECYCLING

Versión	2.0.0
Fecha	15 de Mayo, 2026
Estado	Listo para Producción

1. ROLES E INTEGRANTES

El proyecto fue desarrollado por un equipo de 2 estudiantes, cada uno con funciones clave para asegurar calidad, estabilidad y escalabilidad.

1.1 Joel Saavedra – Team Leader

Responsabilidades principales:

- Dirección general del proyecto
- Diseño arquitectónico
- Supervisión del equipo y roadmap
- Validación final del sistema
- Supervisión de citas y flujo operacional
- Diseño y supervisión operacional de la base de datos (MySQL)
- Refactorizado de la UI

1.2 Monserrat Salazar – Git Master

Nota: "Git Master" se usa como anglicismo técnico para el rol de administradora principal del repositorio. Equivalente a "Administradora del Repositorio".

Responsabilidades principales:

- Administración del repositorio GitHub
- Control de versiones y manejo de branches
- Control de calidad (QA)
- Integración continua (CI/CD)
- Manejo de pipelines de despliegue
- Versionado oficial de releases
- Diseño UI/UX

2. INTRODUCCIÓN

2.1 Visión General

Green Bit es una plataforma web de gestión de reciclaje que conecta a recicladores, recolectores, administradores e instituciones mediante geolocalización, citas inteligentes y un sistema de puntuaciones.

2.2 Objetivos Principales

- Digitalizar y simplificar el proceso de reciclaje.
- Conectar oferta y demanda de materiales en tiempo real.
- Asegurar transparencia con rankings y calificaciones.

- Permitir escalabilidad por institución o ciudad.
- Impulsar prácticas sostenibles.

2.3 Problema que Resuelve

Antes del sistema no existía comunicación directa entre recicladores y recolectores, lo que generaba baja participación y poca transparencia. Green Bit soluciona esto con:

- Conexión instantánea
- Notificaciones en tiempo real
- Rutas optimizadas
- Sistema de rating
- Gestión automatizada de citas

2.4 Stack Tecnológico

- Frontend: React 19, Vite, Bootstrap, Leaflet, Socket.IO.
- Backend: Node.js, Express, MySQL 8+, Socket.IO, Nodemailer.
- Infraestructura: GitHub, MySQL Pool.

3. DESCRIPCIÓN GENERAL DEL SISTEMA

3.1 Funcionalidad General

Green Bit funciona como un marketplace de servicios de reciclaje, permitiendo publicar solicitudes, agendar citas, realizar recolecciones y calificar participantes.

3.2 Flujo Principal

1. Reciclador crea solicitud → OPEN
2. Recolector propone cita → REQUESTED
3. Reciclador acepta/rechaza → ACCEPTED / OPEN
4. Recolección realizada → COMPLETED / CLOSED
5. Ambos califican → Ranking actualizado

3.3 Actores del Sistema

Definición canónica de roles: Reciclador (roleId=3): quien genera residuos y crea solicitudes de reciclaje. Recolector (roleId=2): quien recoge los materiales y propone citas al reciclador.

Actor	Acciones principales
Reciclador	Crear solicitudes, aceptar citas, calificar

Actor	Acciones principales
Recolector	Ver solicitudes, proponer citas, recolectar
Administrador	Gestionar usuarios, materiales y reportes

3.4 Características Clave

- Mapas en tiempo real con ubicación precisa
- Socket.IO para notificaciones instantáneas
- Sistema de ranking y puntaje por usuario
- Gestión de citas y solicitudes
- Control administrativo completo

4. REQUISITOS FUNCIONALES

El sistema cuenta con 22 requisitos funcionales documentados, divididos en módulos. Los requisitos funcionales RF-023 a RF-027, mencionados en versiones preliminares, se encuentran pendientes de documentación formal en el repositorio (ver Doc/API_ENDPOINTS_COMPLETE.md).

4.1 Módulo de Usuarios (5 RF)

RF-001 Registro de Usuarios

Registro de: Reciclador, Recolector, Administrador e Institución. Incluye validación, estado (pendiente/activo/rechazado) y hash de contraseñas.

RF-002 Autenticación

Login con JWT, token de 7 días y cierre de sesión seguro.

RF-003 Recuperación de Contraseña

Envío de código temporal + restablecimiento.

RF-004 Gestión de Perfil

Edición de datos personales, ubicación, contraseña y eliminación de cuenta.

RF-005 Gestión de Instituciones

Asociación de usuarios, aprobación y administración de miembros.

4.2 Módulo de Solicitudes (3 RF)

RF-006 Crear Solicitud

Material, descripción, fotos, ubicación y horarios. Estado inicial: OPEN.

RF-007 Ver Solicitudes Abiertas

Filtros por material, fecha.

RF-008 Actualización de Estado

OPEN → REQUESTED → ACCEPTED → CLOSED.

4.3 Módulo de Citas (4 RF)**RF-009 Programar Cita**

Recolector propone fecha/hora.

RF-010 Aceptar/Rechazar Cita

Aceptada → Solicitud ACCEPTED. Rechazada → Solicitud OPEN.

RF-011 Completar Recolección

Ambas partes confirman → CLOSED.

RF-012 Cancelar Cita

Vuelve la solicitud a OPEN.

4.4 Módulo de Materiales (2 RF)**RF-013 CRUD de Materiales**

Gestión del catálogo por el administrador.

RF-014 Listar Materiales

Filtros, detalles y paginación.

4.5 Módulo de Notificaciones (3 RF)**RF-015 Notificaciones en Tiempo Real**

Eventos: solicitudes, citas, mensajes, rankings.

RF-016 Historial

Listado, filtrado, marcación como leído.

RF-017 Preferencias

Canales, horarios y tipos.

4.6 Módulo de Puntuaciones (2 RF)

RF-018 Calificaciones

1 a 5 estrellas + comentarios.

RF-019 Ranking de Recolectores

Promedios, períodos y TOP 10.

4.7 Módulo de Reportes (3 RF)

RF-020 Dashboard Administrativo

Vista consolidada para el administrador con indicadores clave: total de solicitudes activas, recolectores más activos, materiales más solicitados y estado general del sistema.

RF-021 Reportes

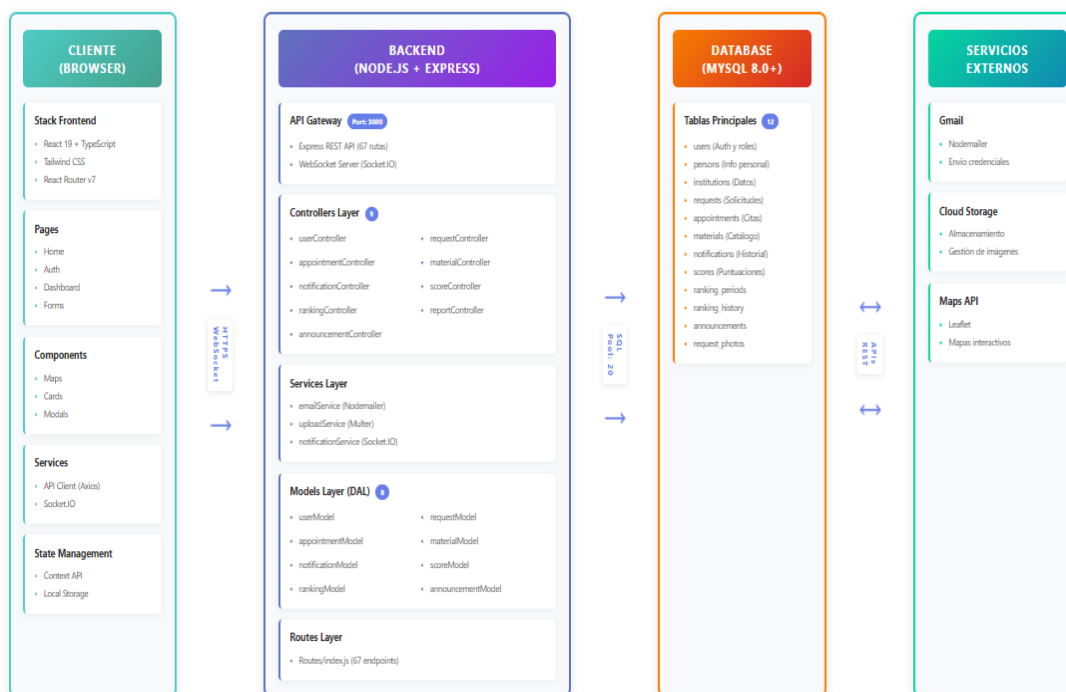
Materiales, recolectores, citas y puntuaciones con gráficos.

RF-022 Exportación

Generación de PDF con branding.

5. ARQUITECTURA DE SOFTWARE

⚠ Contenido pendiente: diagrama de arquitectura y descripción de capas del sistema. Debe incluirse el diagrama de capas (frontend, backend, base de datos) y la descripción de las interacciones entre servicios.



6. ESTRUCTURA DEL PROYECTO

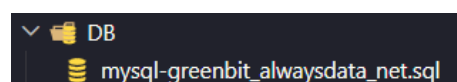
El repositorio sigue una estructura monorepo con las siguientes carpetas principales:

- back/ – API REST con Node.js + Express, lógica de negocio, rutas y servicios.
- front/ – Aplicación web con React + Vite, componentes, vistas y configuración de rutas.
- DB/ – Scripts SQL de inicialización y seed de la base de datos.
- Doc/ – Documentación técnica complementaria (SETUP.md, API_ENDPOINTS_COMPLETE.md, etc.).
- docker-compose.yml / docker-compose.prod.yml – Orquestación de contenedores para desarrollo y producción.

7. Base de Datos

7.1 Roles del Sistema

△ Corrección aplicada: la numeración original presentaba dos subsecciones "7.1". Se unificó con la información de roles consistente con la Sección 3.3.



ID	Rol	Función
1	Admin	Gestión global: aprobaciones, reportes
2	Recolector	Propone citas y ejecuta recolecciones
3	Reciclador	Crea solicitudes de reciclaje

7.2 Datos Clave en la Tabla users

- email: usuario de acceso
- password: hash bcrypt
- roleId: rol asignado
- state: estado del usuario
- registerDate: fecha de registro

Estados principales:

- 3 → Pendiente de aprobación
- 1 → Activo con contraseña temporal
- 2 → Activo con contraseña definitiva
- 0 → Eliminado (no accede)

7.3 Administradores Activos

```
SELECT id, email, phone, state
FROM users
WHERE roleId = 1 AND state != 0;
```

7.4 Exportación Segura (CSV sin contraseñas)

```
SELECT id, email, roleId, state, registerDate
INTO OUTFILE '/secure/admin_users.csv'
FROM users WHERE roleId=1 AND state!=0;
```

7.5 Ciclo de Credenciales

- Usuario creado → contraseña temporal
- Envío de correo
- Primer login → debe cambiar contraseña (state 1 → 2)
- Recuperación → se asigna nuevo password temporal (vuelve a state 1)

7.6 Seguridad Requerida

- Contraseñas de 8 caracteres mínimo
- Rotar JWT secret cada 6 meses

- Auditoría mensual de usuarios inactivos
- MFA recomendado para Admin (futuro)

8. Requisitos del Sistema

8.1 Cliente (Hardware para Computadores)

- CPU 1.6 GHz
- RAM 4–8 GB
- Al menos 500 MB libres
- Resolución mínima 1366×768
- Cualquier dispositivo móvil con acceso a internet y un navegador.

8.2 Cliente (Software)

- Navegadores modernos (Chrome, Edge, Firefox 115+)
- SO: Windows 10+, macOS 12+, Ubuntu 22.04+, Android, iOS

8.3 Servidor (Hardware)

- Producción inicial: 2–4 vCPU
- 16 GB RAM (óptima)
- Almacenamiento 40 GB+
- Red $\geq$ 100 Mbps

8.4 Servidor (Software)

- Ubuntu Server o cualquier host con máquina virtual
- Node.js 18+
- MySQL 8.0
- Nginx con HTTPS
- Docker para montar el contenedor
- PM2 o systemd para procesos
- Backups diarios configurados

8.5 Escalabilidad

- CDN para assets
- Redis para caché
- Migración a una arquitectura API stateless y hexagonal para facilitar el cambio de base de datos.
- Particionado de tabla users si supera la cantidad de registros óptima.

9. Instalación y Configuración

9.1 Clonado

```
git clone https://github.com/RJoel158/PS2-APP-BASURA-espejo
```

9.2 Backend

```
npm install
cp .env.example .env # Configurar variables de entorno
# Crear base de datos según scripts en DB/
npm run dev
```

9.3 Frontend

```
npm install
npm run dev
```

9.4 Pruebas Rápidas

- Backend responde en /health
- Front renderiza login
- Usuario reciclador recibe contraseña temporal
- Cambio de contraseña funciona (state 1 → 2)

9.5 Crear Admin Inicial

```
INSERT INTO users (email, phone, password, roleId, state, registerDate)
VALUES ('admin@greenbit.local', '+0000000000', '$HASH', 1, 2, NOW());
```

9.6 Producción

- Nginx como proxy
- HTTPS habilitado
- PM2 manejando procesos
- Backups diarios y rotación de logs

9.7 Mantenimiento

- Auditoría mensual de estados
- Limpiar carpeta uploads/
- Actualizar dependencias críticas trimestralmente

En su defecto, montar el contenedor de producción desde el repositorio; este deberá levantar las imágenes del frontend, backend y base de datos simultáneamente, siempre y cuando se cuente con las credenciales correctas en los archivos .env.

9.8 Recuperación de Contraseña

```
POST /users/forgotPassword
```

9.9 Checklist Final

- ☐ .env de backend y frontend configurados
- ☐ Base de datos creada
- ☐ Admin inicial cargado
- ☐ HTTPS activo
- ☐ Backups funcionando
- ☐ /health OK

10. Procedimiento de Hosteado / Hosting

- Sitio Web
- Base de Datos
- API / Servicios Web
- Otros (Firebase, etc.)

El hosting puede realizarse con total normalidad tanto en plataformas de terceros como Vercel y Render, como también en una VM servidor proporcionada por la Universidad Privada del Valle haciendo uso de contenedores Docker. Durante el proceso previo a ambos despliegues se realizaron pruebas de rutas y flujos de trabajo en toda la aplicación, de modo que se confirmó que todo debería responder al 100% durante su puesta en marcha.

11. Git

La versión final fue subida con éxito y aprobada tanto por el cliente como por el ingeniero a cargo del proyecto. A continuación se detallan las convenciones y prácticas de control de versiones adoptadas:

- Rama principal: La rama main representa siempre el estado estable y aprobado del proyecto. Todo desarrollo se realiza en ramas derivadas (feature/, fix/, hotfix/) y nunca directamente sobre main.
- Convención de commits: Se utiliza el estándar Conventional Commits (feat:, fix:, docs:, refactor:, chore:), de modo que el historial sea legible y automatizable para changelogs.
- Proceso de merge / Pull Request: Cualquier integración a main debe pasar por un Pull Request revisado y aprobado por la Git Master. No se permiten merges directos sin revisión de código.

- Etiqueta de versión final: La versión de producción entregada está marcada con el tag v1.0.0, siguiendo el estándar Semantic Versioning (MAJOR.MINOR.PATCH).

12. Monitoreo y Logs

El sistema cuenta con las siguientes herramientas y prácticas de monitoreo para entornos de producción:

- PM2 / systemd: Supervisión de procesos del backend, reinicio automático ante fallos.
- Logs de Docker: Consultar con "docker compose logs -f" para revisar la actividad de cada servicio.
- Nivel de logs configurable: Controlado mediante LOG_LEVEL en el archivo .env (valores: info en desarrollo, silent en producción).

13. Personalización y Configuración

13.1 Backend (back/.env)

Parámetros clave:

- PORT → puerto del servidor
- CORS_ORIGIN → URL permitida (frontend)
- JWT_SECRET → clave de seguridad
- EMAIL_* → SMTP para envíos
- MAX_FILE_SIZE / FILE_TYPES → control de uploads

13.2 Frontend (front/.env)

- VITE_API_BASE_URL → URL del backend
- Configuración del mapa: centro, zoom
- Límites UI: VITE_ITEMS_PER_PAGE

13.3 Gestión de Roles

API:

```
PUT /users/:id/role
```

Frontend: control mediante ProtectedRoute.

13.4 Reportes

Activar filtrado por usuario: userId en endpoints.

14. Seguridad

14.1 Autenticación

- Contraseñas con bcrypt
- JWT con expiración y secreto fuerte

14.2 Autorización

- Acceso basado en roleid y state
- Rutas admin deben estar protegidas

14.3 Buenas Prácticas

- HTTPS obligatorio
- CORS restringido
- Rotación periódica de secretos
- Backups y firewall activo
- Rate limiting para evitar ataques y ELO boosting en los rankings

14.4 Datos Sensibles

El .env siempre en .gitignore. Jamás exponer:

- Hashes
- Contraseñas temporales
- Tokens internos

15. Depuración y Solución de Problemas

15.1 Conexión a BD

- Verificar MySQL activo
- Revisar credenciales en .env
- Probar endpoint: GET /health

15.2 Errores CORS

- Comprobar CORS_ORIGIN
- Asegurar coincidencia exacta del dominio y puerto

15.3 Problemas con Subidas

- Permisos de carpeta uploads/
- Tamaño y tipos permitidos en .env

15.4 Fallos de Login

- Revisar email/contraseña
- Verificar state: 1 = debe cambiar contraseña, 2 = activo
- Ver logs del backend

15.5 Envío de Email

- Revisar EMAIL_*
- Asegurar "App Password" en Gmail u otros proveedores

16. Glosario de Términos

Término	Definición
Admin	roleId=1, control total del sistema
Recolector	roleId=2, propone citas y ejecuta recolecciones
Reciclador	roleId=3, genera residuos y crea solicitudes de reciclaje
state	0 eliminado, 1 inicial (contraseña temporal), 2 activo, 3 pendiente
Soft delete	No borra el registro, solo lo desactiva (state=0)
Credencial temporal	Contraseña automática generada para el primer acceso
Marketplace	Plataforma que conecta oferta y demanda de servicios de reciclaje
Backend	Capa de servidor: API REST en Node.js + Express
Frontend	Capa de cliente: aplicación web en React + Vite
Socket	Conexión persistente bidireccional para notificaciones en tiempo real (Socket.IO)
Hash	Resultado del cifrado de contraseñas mediante bcrypt

17. Referencias y Recursos

Documentación interna del repositorio:

- Doc/SETUP.md
- API_ENDPOINTS_COMPLETE.md
- ROUTES_CONSOLIDATION_COMPLETE.md

Recursos externos:

- Express – <https://expressjs.com/>
- MySQL – <https://dev.mysql.com/doc/>
- React Router – <https://reactrouter.com/>
- Bootstrap 4 – <https://getbootstrap.com/docs/4.6/>

18. Herramientas de Implementación

Frontend

- JavaScript / TypeScript
- React + Vite

Backend

- Node.js + Express

Bibliotecas principales

Biblioteca	Función
Axios	Cliente HTTP
Leaflet	Mapas interactivos
Multer	Gestión de uploads
Nodemailer	Envío de correos
bcrypt	Hashing de contraseñas
mysql2	Conexión a base de datos
Socket.IO	Notificaciones en tiempo real

19. Bibliografía y Créditos

Fuentes:

- MDN Web Docs
- Documentación oficial de React y Express
- Stack Overflow
- Diversos agentes personalizados de IA, skills construidas durante el desarrollo de la aplicación web, proporcionados por GitHub Copilot, Ollama, Anthropic y Google.